

Group 03 - Project Report

Advanced Analytics and Applications

Team 03



Authors:

Angela Feng (7397232)

Moritz Lang (7430881)

Beate Kranz (7435471)

Lennart Jekel (7444132)

Contact: ljekel1@smail.uni-koeln.de

Supervisor: Univ.-Prof. Dr. Wolfgang Ketter

Co-Supervisor: Janik Muires

Chair of Information Systems for Sustainable Society
Faculty of Management, Economics and Social Sciences
University of Cologne

2026-07-26

Table of contents

1 Problem Description	1
2 Data Description	1
3 Data Preparation	1
3.1 Summary of the Central Data-Preparation Steps	2
4 Data Analysis NN	3
4.1 SVM	4
4.2 SVR	5
4.3 Results	5
4.4 SVC	5
4.5 Results	5
4.6 How spatial resolution seems to change the outcome?	5
4.7 How temporal resolution changed the outcome?	5
5 Spatial Encoding	5
6 SVR	5
7 SVC	5
8 Comparison	5
9 Smart Charging Reinforcement Learning	5
9.1 Motivation and Setup	5
9.2 Results	6
9.3 Implications	6
10 Conclusion	7
11 Data Description	7
11.1 Prototyping in Notebooks	7
12 Appendix	8
12.1 Neural-network baseline training configuration	8
12.2 Neural-network baseline validation results	9
13 Citations and References	9

List of Figures

1 A sample plot generated directly in the report.	7
---	---

1 Problem Description

business goal and data mining goal

position of a top-tier management consultancy (Data Science Team of Bane & Wayne Partners (BWP))

Business Background: A renowned German car company is in the process of establishing a platform for ride-hailing mobility services. Initially, the client wants to offer its US customers (as customers from the US market are more receptive for novel business models) a ride-hailing platform using a fully electrified vehicle fleet. However, the customer lacks tactical and strategic know-how in the area of operations management of electrical vehicle fleets. Therefore, our client asks for our help

and would like to better understand the dynamics in the field of ride-hailing for a US city in temporal and spatial resolution.

Taxis are considered a good-enough proxy for potential ride hailing vehicles.

Also, provide a detailed description of the trip dataset such that there are no pending questions.

2 Data Description

Analyze taxi demand patterns: Specifically show how these patterns (start time, trip length, start and end location, price, average idle time between trips, and so on) for the given sample varies in different spatio-temporal resolution (i.e., census tract vs. varying hexagon diameter and/or temporal bin sizes). Give possible reasons for the observed patterns. Furthermore, can you identify spatial hot spots for trip demand using Gaussian Mixture Models (i.e., using Spatial Kernel Density Estimation)?

3 Data Preparation

Task: “ “ ” in other words, your method should predict for each spatial unit (in whatever spatial layer of analysis you decide on, e.g., hexagon or census tract) and time-basket (e.g., 08am-11.59am) the taxi demand. Important: It is not your goal to predict hour $x+1$ demand given hour x demand, rather to produce an answer to questions like “what demand can we expect on a rainy Sunday from 10 – 11 am around the outskirts of the city”. Also advise a reasonable validation strategy for your prediction model (i.e., definition of test, training data etc). “ “ ”

- Data sources: Taxi data, weather data, Census tracts, community areas, Point of interest data
- Taxi data was downloaded from the Chicago Data Portal via API
- Full run is from first january 2024 to first may 2026

- The sample run uses one reproducible, contiguous 14-day window (2026-04-13 through 2026-04-26, inclusive). This preserves complete weekdays, weekends, and daily demand cycles while keeping notebook runtimes short. A 400,000-trip cap is only a safety guard; reaching it is treated as an invalid, potentially truncated sample.
- Initially a data profiling was conducted to better understand the data, find null values and plausibility issues and coverage of the Data
- This data profiling layed down the basis for the following data-prep steps
- Data was prepared following a simple bronze-silver-gold pipeline logic, whereby each dataset runs through the levels:
- Bronze represents the raw data stored as parquet file
- Silver adjusted the data quality, missing values, inadequate data, consistency checks
- Gold aggregates data, joins different data sets
- Regarding gold / generating of the temporal-spatial dataset
- First we created a dataframe containing the temporal unit, i. e. 1h, 2h or 4h. This dataframe contained nothing but the timestamps
- We integrated the weather data to this dataframe and also holidays
- Next we merged the temporal dataframe with the spatial units, i. e. census tracts, hexagons and community areas
- This gave us the keys of the final dataset and we could aggregate the taxi data into this dataframe

Thereby creating certain data characteristics that become problematic and need to be addressed during predictive tasks: - Most Temporal-Spatial units contain no or only very small demand (e. g. 3 am in some outer area of chicago) while only few contain high demand - Therefore we decided to use community area as main spatial unit

3.1 Summary of the Central Data-Preparation Steps

- Load the taxi, weather, spatial-boundary, and OpenStreetMap point-of-interest data and store the raw inputs as Parquet files in the bronze layer.
- Profile the raw data for missing values, duplicates, temporal coverage, and implausible trip, fare, and location values; use these findings to define the subsequent cleaning rules.
- Create the silver taxi dataset by removing duplicate or implausible trips, requiring valid pickup information, and retaining trips with positive fares and totals, non-negative cost components, distances of more than 0 and at most 250 miles, and durations between 1 minute and 3 hours.
- Restrict the weather observations to the taxi-data period, repair the census-tract and community-area geometries, standardize their identifiers, and generate an H3 grid covering Chicago.
- Build the gold layer by assigning taxi trips and POIs to H3 cells, census tracts, and community areas and by excluding pickup locations outside the Chicago grid.

- Construct complete 1-, 2-, and 4-hour time grids and enrich them with aggregated and imputed weather data, Illinois holiday indicators, and cyclical month, weekday, and hour features.
- Cross the temporal grids with every spatial unit, add POI counts by category, and aggregate taxi pickups as `trip_count`; explicitly retain spatial-temporal combinations with zero demand.
- Produce comparable demand datasets for all temporal and spatial resolutions, with community areas used as the principal modeling unit because finer resolutions contain a particularly high proportion of sparse or zero-demand observations.
- Create reproducible train, validation, and test datasets (approximately 70/15/15), grouping complete calendar days into the same split and checking that no dates leak across the partitions.
- 4 Datasets in silver:
 - Community area: complete data set as community area is always as spatial unit given
 - Census tracts: is more granular than community area and not always given (number of missing values), therefore we remove data without `census_tract` here
 - Hexagon: the hexagons are build on the `census_tracts` and therefore only reasonable when we have `census_tract` data
 - community area filtered: to make a fair comparison between the spatial units we create a filtered dataset of community area => Thereby we have one main dataset on community area and for a fair comparison of spatial units we have 3 filtered datasets

4 Data Analysis NN

- Baseline prediction
 - Zero prediction
 - Avg Spatial x Time x Weekday
 - Global median
- Target and Features
 - We selected “`trip_count`” as target for the demand, as this will tell us how many taxi trips were started in one spatial time unit
 - Features: cyclic encoding of hour, weekday and month; weather data; holiday flag and POI information; all other taxi related columns were removed as it would otherwise lead to leakage
- Preprocessing steps:
 - Spatial Encoding is needed as the spatial units are otherwise interpreted as continuous feature
 - `StandardScaler` for the `feature_cols` fitted on training data only
- First Baseline NN
 - Model Architecture in
 - First we trained a very simple NN on all filtered datasets
 - The model architecture is has two hidden layer, the first with 64 Neurons and the second with 32 Neurons.

- As activation function we use ReLu and the output layer uses Softplus function to make all predictions ≥ 0
- Loss Function: MSE
- Selection Criteria of best model: MAE
- with Adam as Optimizer and a learning rate of 0.001
- we uses seed 42 for reproduceability
- The validation results of all neural-network baseline models are presented in.

How did we handle the problem of sparse temporal spatial data: - we decided to keep them, as they reflect actual data and removing them would distort the data - weights - Loss-Function - selection_score

Model, Architecture and Training v5 introduces embedding of community_area

Grid Search for Hyperparameters:

Relatively small search space (can be justified because we already trained some NNs):
`search_space = { "dropout_rate": [0.0, 0.1, 0.2], "weight_decay": [0.0, 1e-6, 1e-5, 1e-4], "lr": [1e-3, 5e-4], "loss_beta": [0.25, 0.5, 1.0], "high_weight": [2.0, 3.0, 5.0, 8.0], }`

Advanced MAE Performance Measure:

E. g.: `selection_score = overall_mae + alpha * high_demand_mae`

More advanced NN model:

- 2 NNs one for high, one for low demand: 1. Classifier predicts probability for high demand
2. final prediction = `p_high * high_model_pred + (1 - p_high) * low_model_pred`

4.1 SVM

- changes we made during the developement of th svm in steps and why we decided on them
- started with a basic svr on rbf kernel which is the basic kernel for scikits() implementation of svms
- added grid search to find best hyperparameter
- added scaler
- changed grid search to only include the hyperparameter relevant for each kernel
- added nystroem
- changed gridsearchcv to randomsearchcv to make hyperparameter fitting quicker
- changed gridsearch to include standardscaler in pipeline instead of manually
- added transgressor
- added memory
- for linear use linear
- change available hyperparamter through testing
- changed to halving... instead of randomsearchcv due to better results
- tested this out as the last option due to still being experimental
-
- due to the data having a lot of $y=0$ the grid search complains about convergance issues -> this is another reason why svm for this particaluar data with h3 is not the best method
- The time frames and spatial units, used above in the Neural Network() were used here as well.
- These include time frames of one, two and twentyfour hours.
- The used spatial units include

4.2 SVR

4.3 Results

4.4 SVC

4.5 Results

4.6 How spatial resolution seems to change the outcome?

4.7 How temporal resolution changed the outcome?

SVM

5 Spatial Encoding

6 SVR

7 SVC

8 Comparison

9 Smart Charging Reinforcement Learning

9.1 Motivation and Setup

The daily operation of electric vehicles requires them to recharge daily. Depending on the time of day, charging costs can vary significantly, especially between peak and off-peak hours. Therefore, charging cost is a significant operational expense for an electric fleet.

A naive strategy of always charging at maximum power ensures the battery is full, but comes at an unnecessarily high cost since the cost grows exponentially with power. However, if we assume the vehicle is home for a fixed window, for example between 2pm and 4pm, we can be smart about when and how much to charge. With reinforcement learning we can simulate this smart charging behaviour, where an agent learns to balance cost savings against the risk of running out of energy.

We model this as a Markov Decision Process (MDP), where the agent observes the current time and battery level, picks a charging power level, and receives feedback in the form of a cost signal.

Our MDP is defined as follows:

- State: the current 15-minute timestep (2:00pm, 2:15pm, ..., 4pm) and the current battery level.
- Action: one of 4 discrete charging power levels: 0 kW (off), 7 kW (low), 14 kW (medium), 22 kW (high).
- Transition: the battery increases by the chosen power $\times 0.25\text{h}$, capped at battery capacity.
- Reward: negative charging cost each step. At the final step (4pm), a random energy demand is drawn from a normal distribution ($\mu = 30 \text{ kWh}$, $\sigma = 5 \text{ kWh}$). If the battery cannot cover this demand, a large penalty is applied.

We made the following assumptions, grounded in real-world data:

- Battery capacity: 60 kWh; Tesla Model Y is one of the most used EV taxi models in North America, and 60 kWh represents a typical usable battery capacity for current EVs.
- Charging levels: 0, 7, 14, 22 kW; Level 2 home chargers support up to 22 kW, which is sufficient for the Tesla Model Y.
- Starting battery at 2pm: random between 5 and 20 kWh; the vehicle arrives partially depleted after a morning shift.
- Cost coefficients: 0.052–0.076; calibrated to match real Chicago home charging rates of \$0.11–\$0.16 per kWh, with the coefficient increasing linearly across timesteps to reflect time-of-use pricing.

We solve this using Q-Learning, a standard reinforcement learning method that learns optimal decisions through trial and error. Q-Learning is well suited here because the state space is small and discrete, allowing the agent to learn a complete policy without the overhead of a neural network.

We additionally implemented a Deep Q-Network (DQN) to test whether a neural network-based approach could match or improve on the tabular solution. While Q-Learning works well for our small state space, DQN would scale better to more complex scenarios, for example if the charging window were longer or if real-time electricity prices were included as an additional state variable.

9.2 Results

We evaluated the learned policies over 20000 simulated episodes against a naive baseline that always charges at maximum power (22kW). The results are summarized below: Policy Avg Cost Savings Avg Battery Fail Rate

Baseline (always max)	10.14	0.0%	55.9	0.0%	Q-Learning	6.38	37.1%	47.1	0.0%	DQN	8.24	18.8%	50.2	0.0%
-----------------------	-------	------	------	------	------------	------	-------	------	------	-----	------	-------	------	------

The Q-Learning agent achieves a 37.1% cost reduction compared to the baseline while maintaining a 0% out-of-energy rate. It does this by charging more aggressively during the early, cheaper timesteps and reducing or skipping charging later when cost coefficients are higher. The average battery departure (47.1 kWh) comfortably exceeds the mean demand of 30 kWh, providing a safety margin against demand uncertainty without overcharging.

The DQN agent also outperforms the baseline with an 18.8% cost reduction, though it charges more conservatively than Q-Learning, resulting in a higher average battery at departures (50.2kWh). This is expected since the tabular Q-Learning approach can represent the optimal policy exactly for our small, discrete state space. Meanwhile the DQN introduces approximation, however, its advantage lies in its scalability, as it could handle a more complex scenario with continuous state variables or a longer charging window without modification.

All three policies achieve a 0% out of energy rate, confirming that the penalty mechanism successfully teaches the agents to prioritize energy sufficiency. This is substantial, since this could leave the driver stranded and lose a full day of revenue.

9.3 Implications

The results demonstrate that even in a simplified two-hour charging window, an intelligent charging agent can reduce costs by over a third compared to a naive strategy. Scaled to an entire fleet, these savings become significant. For example, if a fleet of 100 vehicles each charges once per day, a 37% reduction in per-vehicle charging cost translates directly into lower daily operating expenses.

The smart charging system is fully automated and requires no manual intervention from drivers or dispatchers. Once trained, the agent makes real-time charging decisions based on the current battery level and time of day, adapting naturally to varying starting conditions.

Moreover, the approach is flexible. If the electricity pricing structures change or the charging window shifts or the parameters are changed, the agent can simply be retrained on the updated parameters. This makes it a future-proof component of the fleet's operational toolkit, complementing the demand prediction models developed in earlier sections of this analysis.

10 Conclusion

Template

11 Data Description

In this section, we demonstrate how to include analysis results in your report. You can either write code directly in this `.qmd` file or reference results from a separate prototyping notebook.

11.1 Prototyping in Notebooks

You can work on your analysis in the `notebooks/prototyping.ipynb` file. When you are ready to include a result, you can use Quarto's `embed` shortcode. This is a powerful feature that allows you to maintain a clean report while keeping your extensive prototyping code in separate notebooks.

To do this, you must label the specific cell in your notebook using `#| label: fig-some-name` (and optional `#| fig-cap: ...`). Then, you can embed it here like this:

Note that this requires your `.ipynb` notebooks to follow Quarto's cell-metadata syntax.

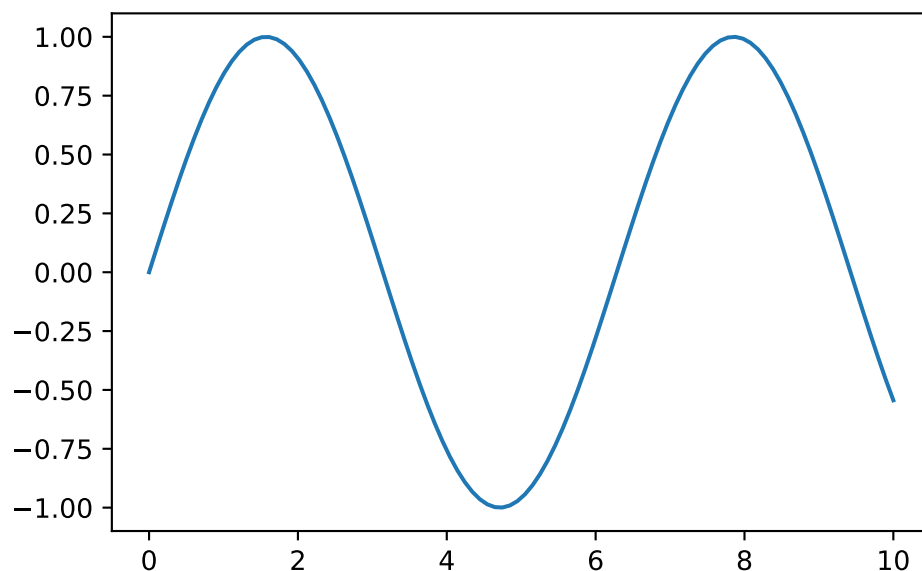


Figure 1: A sample plot generated directly in the report.

In the PDF version, the code above is hidden by default to keep the report concise and focused on the results. In the HTML version, the code can be expanded using the “Code” button.

12 Appendix

Table 1: Architecture of the neural-network baseline

Component	Input dimension	Output dimension	Activation
Numeric input	n_{features}	n_{features}	–
Hidden layer 1	n_{features}	64	ReLU
Hidden layer 2	64	32	ReLU
Numeric output	32	1	Linear
Spatial bias	Spatial-unit ID	1	–
Combination	Numeric score + spatial bias		1 Addition
Final output	1	1	Softplus

The spatial bias is implemented as a one-dimensional embedding. It represents an additive spatial intercept rather than a multidimensional representation-learning embedding. Index zero is reserved for spatial units not observed in the training split and has no learned effect.

12.1 Neural-network baseline training configuration

Table 2: Training configuration of the neural-network baseline

Setting	Configuration
Datasets	H3 resolution 7, census tracts, and filtered community areas
Temporal units	1h, 4h, and 24h
Number of models	9
Training objective	Mean squared error (MSE)
Optimizer	Adam
Learning rate	0.001
Batch size	1,024
Maximum epochs	100
Early-stopping patience	10 epochs
Minimum improvement	0
Checkpoint-selection metric	Validation MAE
Output constraint	Softplus activation
Numerical preprocessing	StandardScaler fitted exclusively on TRAIN
Spatial preprocessing	Categorical mapping fitted exclusively on TRAIN
Reproducibility	Dataset-specific seed, starting at 42
Model development split	TRAIN for fitting and VAL for model selection
Final evaluation split	TEST remains unused during model development

12.2 Neural-network baseline validation results

```

from pathlib import Path

import pandas as pd

results_path = Path( "models/full/baseline_NN_spatial_bias_v1_validation_results.parquet" )
if not results_path.exists(): raise FileNotFoundError( f"Missing baseline results: {results_path}" )
" " "Run the NN-baseline notebook in Full Mode before rendering the report."
)

baseline_results = ( pd.read_parquet( results_path, columns=[ "model_tag",
"nn_val_mae", "spatial_time_weekday_mean_val_mae", "nn_val_mae_skill_score", ], )
.sort_values("nn_val_mae_skill_score", ascending=False) .rename( columns={ "model_tag":
"Model", "nn_val_mae": "NN MAE", "spatial_time_weekday_mean_val_mae": "Naive
baseline MAE", "nn_val_mae_skill_score": "MAE skill score", } ) .round(4) )

baseline_results " "

```

13 Citations and References

You can easily cite your sources using BibLaTeX. For example, you can cite the Quarto documentation (Team, 2024) or a textbook like “R for Data Science” (Wickham et al., 2023). The bibliography will be automatically generated at the end of the document.

References

- Team, Q. (2024). Quarto: An open-source scientific and technical publishing system. *Journal of Modern Data Science*. <https://quarto.org>
- Wickham, H., Çetinkaya-Rundel, M., & Grolemund, G. (2023). *R for data science*. O’Reilly Media.